



INTERNSHIP LOGBOOK

Exploring quantization strategies for advanced neural-network architectures as ansätze in variational quantum Monte Carlo (VQMC).



Author: Marta Casado Carrasquer
Email: martacc01@gmail.com
Semester: Spring 2026
Area: Physics
Course code: FK8026

Contents

1	Internship Information	1
2	Objectives	1
3	Week 1	2
3.1	Monday, June 15, 2026	2
3.2	Tuesday, June 16, 2026	2
3.3	Wednesday, June 17, 2026	2
3.4	Thursday, June 18, 2026	2
3.5	Friday, June 19, 2026	6
4	Week 2	6
4.1	Monday, June 22, 2026	6
4.2	Tuesday, June 23, 2026	7
4.3	Wednesday, June 24, 2026	8
4.4	Thursday, June 25, 2026	11
4.5	Friday, June 26, 2026	15
5	Week 3	15
5.1	Monday, June 29, 2026	15
5.2	Tuesday, June 30, 2026	15
5.3	Wednesday, July 01, 2026	18
5.4	Thursday, July 02, 2026	19
5.5	Friday, July 03, 2026	20
6	Week 4	21
6.1	Monday, July 06, 2026	21
6.2	Tuesday, July 07, 2026	21
6.3	Wednesday, July 08, 2026	23
6.4	Thursday, July 09, 2026	23
6.5	Friday, July 10, 2026	23
7	Week 5	24
7.1	Monday, July 13, 2026	24
7.2	Tuesday, July 14, 2026	24
7.3	Wednesday, July 15, 2026	25
7.4	Thursday, July 16, 2026	26
7.5	Friday, July 17, 2026	27
8	Week 6	28
8.1	Monday, July 20, 2026	28

1 Internship Information

- **Institution:** Instituto de Nanociencia y Materiales de Aragón (INMA).
- **Supervisors:** Jesús Carrete Montaña & David Zueco Láinez.
- **Period:** 15th June 2026 – 20th July 2026.
- **Project title:** Exploring quantization strategies for advanced neural-network architectures as ansätze in variational quantum Monte Carlo (VQMC).

2 Objectives

The main objective of this internship was to investigate the use of numerical quantization techniques for neural-network quantum states (NQS) used as variational ansätze in Variational Quantum Monte Carlo (VQMC). In particular, the internship aimed to explore whether reducing the numerical precision of neural-network parameters and computations could decrease the computational cost while preserving the accuracy and physical properties of the resulting quantum states.

3 Week 1

3.1 Monday, June 15, 2026

Activities

- Initial meeting with supervisors.
- Discussion of internship objectives and expected outcomes.
- Installation and setup of the working environment: create a GitHub repository for the project, set a Python environment and install JAX and NetKet.
- Familiarize myself with the NetKet library by trying out its tutorials.

Concepts Learned

- Learnt to manage Python projects with uv.
- Started learning to work with GPUs.

3.2 Tuesday, June 16, 2026

Activities

- Exploration of the reference data and its working code.
- Cloned the repository and executed example simulation for different parameters.
- Resolved execution issues.

Concepts Learned

- Architecture of working code and reference data.

3.3 Wednesday, June 17, 2026

Activities

- Produced table and figures from data obtained by running the test simulation for different parameters.
- Compared generated results with figures from the reference paper.

Concepts Learned

- Valuable metrics such as: Renyi-2 entropy, V-score, staggered magnetization or fidelity.

3.4 Thursday, June 18, 2026

Activities

- Produced table and figures from data obtained by running the test simulation for different parameters.
- Compared generated results with figures from the reference paper.

Concepts Learned

- Evaluation of the obtained metrics:

We first got the functions and parameters needed to perform our simulation from the existing working code.

We then performed a two-dimensional parameter sweep over the coupling space (α, J) of the long-range quantum Hamiltonian. For each parameter combination, we computed key observables. Exact

Diagonalization (ED) was used as a reference to validate the neural-network results and study its ability to learn highly entangled quantum states.

Table 1 below, shows 6 physical points from the chosen range of α and J values (based on FIG 4 (a) from the reference data), where the 2 first points correspond to the ferromagnetic phase, another 2 to the paramagnetic phase and the final 2 points correspond to the antiferromagnetic phase.

Table 1: Computation of key observables for the ferro, para and antiferromagnetic phases.

N	α	J	b	demb	h	$\langle \hat{E} \rangle$	V-score	$\langle \hat{S}_2 \rangle$	$\langle \hat{m}^2 \rangle$	$\langle \hat{m}_s^2 \rangle$
10	6.0	-5.0	2	16	2	-2.655270005178951	0.003469	0.979202	0.900859	0.010078
10	4.0	-3.0	2	16	2	-1.7641913821094541	0.000073	0.983389	0.595469	0.032344
10	1.0	1.0	2	16	2	-0.9175663015385557	0.002543	-0.006853	0.072422	0.097891
10	4.0	1.0	2	16	2	-0.8670312829124209	0.001099	0.026948	0.079297	0.138203
10	4.0	7.0	2	16	2	-1.1140409653913692	0.040701	1.071040	0.008828	0.904609
10	2.5	9.0	2	16	2	-1.0443864652545338	0.095450	1.191378	0.004219	0.953594

Table 1 shows a spin system of size $N = 10$ where:

$\langle \hat{E} \rangle$ is the Expectation Value of Energy per Site.

It gives us the ground state energy expectation value per spin of the quantum Hamiltonian. It is the primary target in Variational Monte Carlo (VMC) since according to the variational principle, a lower energy value indicates a state closer to the true ground state.

From the table we observe:

Ferromagnetic Phase ($J < 0$): Has the lowest energy values (≈ -2.65 and -1.76) because the strong coupling J stabilizes the aligned spin configuration.

Paramagnetic/Antiferromagnetic: Energies shift higher (≈ -0.86 to -1.11) as interaction parameters change.

V-score is a normalized measure of energy variance:

$$\text{V-score} = \frac{\text{Var}(\hat{H})}{|\langle \hat{E} \rangle|^2} = \frac{\langle \hat{H}^2 \rangle - \langle \hat{H} \rangle^2}{|\langle \hat{E} \rangle|^2}$$

It serves as a quality metric for how close your neural network ansatz is to being a true eigenstate of the Hamiltonian. For an exact ground state, energy variance is strictly 0, making V-score = 0.

From the table we observe:

Low V-scores (10^{-5} to 10^{-3}). The model learned the ground state extremely well in the ferromagnetic and paramagnetic regions (e.g., row 2 with V-score ≈ 0.000073).

High V-scores (0.04 to 0.09). In the antiferromagnetic phase (rows 5 & 6), the V-score increases significantly. This indicates that the state is more complex, making it harder for the network to reach exact convergence.

$\langle \hat{S}_2 \rangle$ is the Second-Order Renyi Entanglement Entropy.

It is a measure of quantum entanglement between a subsystem and the rest of the system: $S_2(\rho_A) = -\ln \text{Tr}(\rho_A^2)$. It quantifies the degree of non-local quantum correlations present in the state.

From the table we observe:

Ferromagnetic/Paramagnetic ($\langle \hat{S}_2 \rangle \approx 0.98$ to 0.02): low or moderate entanglement.

Antiferromagnetic ($\langle \hat{S}_2 \rangle > 1.0$): entanglement reaches its highest values (≈ 1.07 to 1.19), indicating a highly entangled quantum state.

$\langle \hat{m}^2 \rangle$ is the Squared Magnetization.

It is the square of the total magnetization per site:

$$\hat{m} = \frac{1}{N} \sum_i \hat{\sigma}_i^z$$

It is the order parameter for ferromagnetism, detecting if spins align parallel to each other.

From the table we observe:

Ferromagnetic phase: very high (≈ 0.90 and 0.59), confirming strong parallel alignment of spins.

Paramagnetic & Antiferromagnetic phases: drops close to 0 (≈ 0.004 to 0.07), indicating no global uniform alignment.

$\langle \hat{m}_s^2 \rangle$ is the Squared Staggered Magnetization.

It is the order parameter for antiferromagnetism:

$$\hat{m}_s = \frac{1}{N} \sum_i (-1)^i \hat{\sigma}_i^z$$

It checks whether adjacent spins align antiparallel to each other.

From the table we observe:

Ferromagnetic/Paramagnetic phases: suppressed close to zero (≈ 0.01 to 0.13).

Antiferromagnetic phase: dominates near maximum capacity (≈ 0.90 and 0.95).

In order to observe how the squared magnetization $\langle \hat{m}^2 \rangle$ and the squared staggered magnetization $\langle \hat{m}_s^2 \rangle$ change as we go from the ferromagnetic to the antiferromagnetic phase, we obtained the following graph:

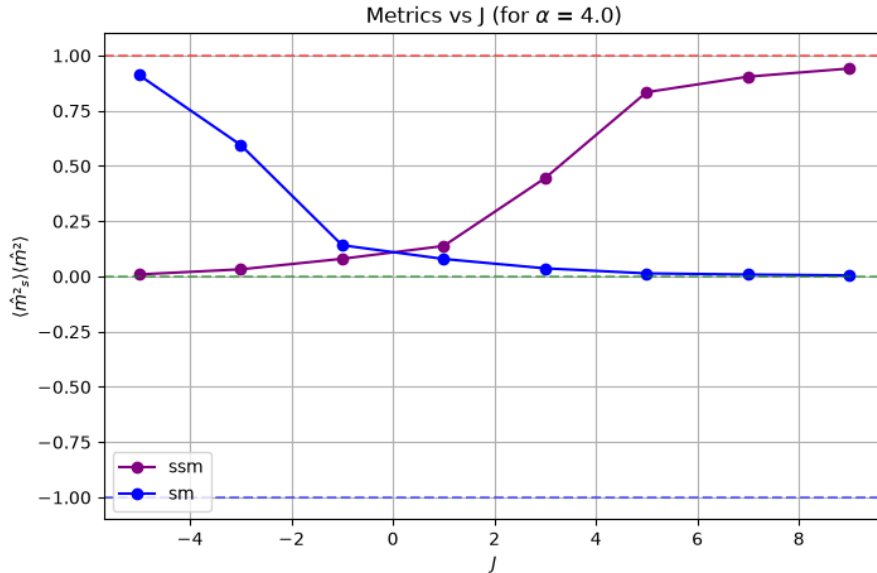


Figure 1: Evolution of the ssm ($\langle \hat{m}_s^2 \rangle$) and the sm ($\langle \hat{m}^2 \rangle$) over J .

A comparison between the variational ground state wave function learned by the Vision Transformer (ViT) and the exact ground state from Exact Diagonalization was also made (Figure 2).

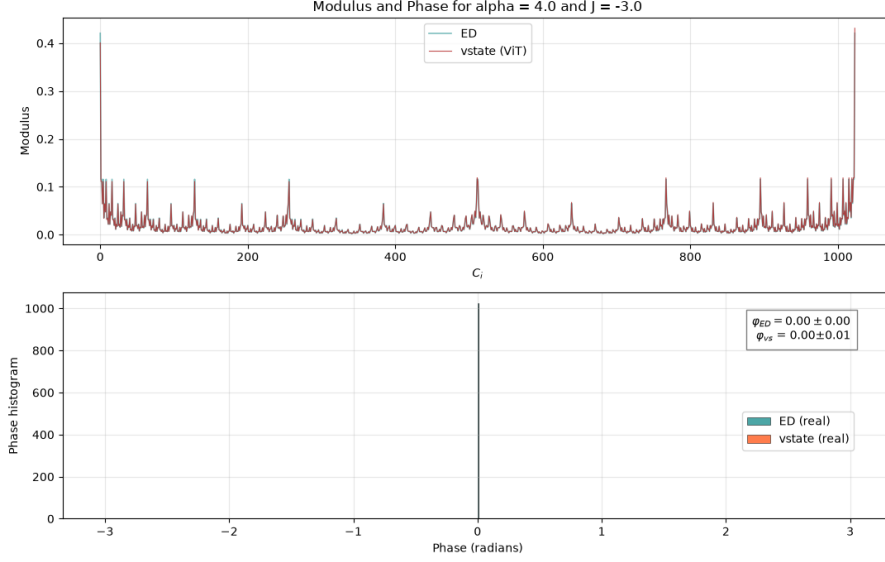


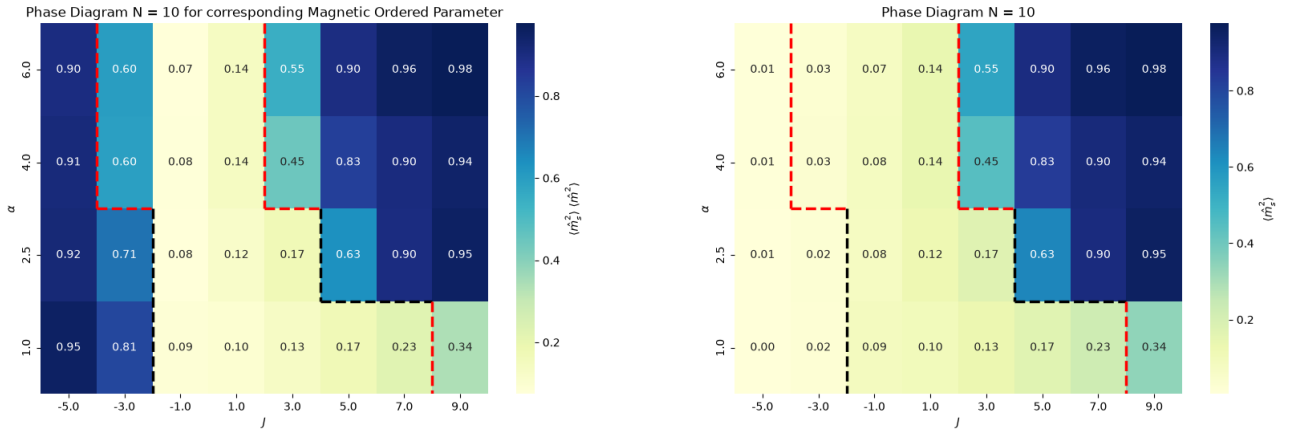
Figure 2: Modulus and Phase histogram representation for the ferromagnetic phase.

Looking at the image above we see that for the wavefunction amplitude we got a near perfect modulus match. The red curve (vstate) and blue curve (ED) overlap almost completely across all $2^{10} = 1024$ Hilbert space basis configurations (C_i).

We can clearly identify that we are in the ferromagnetic phase by noticing the two massive sharp peaks at the extreme boundaries: $C_0 = |0000000000\rangle = |\uparrow\uparrow \dots \uparrow\rangle$ and $C_{1023} = |1111111111\rangle = |\downarrow\downarrow \dots \downarrow\rangle$. These correspond to the two fully polarized ground state configurations of the ferromagnetic phase. The neural network correctly identifies that these two configurations carry the vast majority of the probability amplitude (≈ 0.42).

For the wavefunction phase structure we find a zero phase difference. The phase histogram shows a single, extremely sharp delta-like spike at Phase = 0.00 radians for all 1024 basis components. In the ferromagnetic region ($J < 0$), the quantum ground state coefficients are purely real and non-negative ($\psi(C_i) > 0$). The results obtained: $\varphi_{ED} = 0.00 \pm 0.00$ and $\varphi_{vs} = 0.00 \pm 0.01$, prove that the Vision Transformer learned the trivial sign structure without introducing numerical noise.

Furthermore, we generated phase diagrams across the (α, J) parameter space but using much lesser points than FIG 4 (a) from the reference data.



(a) Phase diagram measuring $\langle \hat{m}_s^2 \rangle$ and $\langle \hat{m}^2 \rangle$.

(b) Phase diagram measuring $\langle \hat{m}_s^2 \rangle$.

Figure 3: Ground state phase diagrams across (α, J) for corresponding magnetic order parameters.

Figure 3a measures a combined order parameter ($\langle \hat{m}^2 \rangle$ for $J < -1.0$ and $\langle \hat{m}_s^2 \rangle$ for $J \geq -1.0$) to capture both ferromagnetic and antiferromagnetic ordering simultaneously while Figure 3b only measures the Squared Staggered Magnetization $\langle \hat{m}_s^2 \rangle$.

For the ferromagnetic phase ($J < -1.0$) we find that in Figure 3b, staggered magnetization ($\langle \hat{m}_s^2 \rangle \approx 0.00 - 0.03$) completely vanishes because spins are aligned in parallel. However, switching to uniform magnetization $\langle \hat{m}^2 \rangle$ in Figure 3a reveals values up to 0.95, confirming a strongly ordered ferromagnetic phase. We also observe that as $\alpha \rightarrow 1.0$ (stronger long-range interactions), the ferromagnetic order parameter increases ($0.90 \rightarrow 0.95$) since long-range interactions enhance parallel alignment.

For the paramagnetic phase ($-1.0 \leq J \leq 1.0$) we find that for both Figure 3a and Figure 3b the order parameters collapse to near-zero ($\approx 0.07 - 0.14$). That is because quantum fluctuations dominate over magnetic coupling, destroying both parallel and antiparallel long-range order.

Finally, for the antiferromagnetic phase ($J \geq 3.0$) we observe how in both Figure 3a and Figure 3b, $\langle \hat{m}_s^2 \rangle$ rises toward 0.98 as J increases. For long range interactions (α), at short range limits ($\alpha = 6.0$), strong antiferromagnetism sets in quickly at lower coupling ($J \geq 3.0 \rightarrow 0.55$). At strong long range limits ($\alpha = 1.0$), long range interactions create severe frustration for antiferromagnetic alignment. As a result, $\langle \hat{m}_s^2 \rangle$ drops drastically to 0.34 even at $J = 9.0$.

3.5 Friday, June 19, 2026

Activities

- Learn about the Mushroom dataset from the UCI Machine Learning Repository.
- Become familiar with the JAX environment and the Flax Linen neural network library.

Concepts Learned

- Learnt about binary classification and how the Mushroom dataset works.
- Started to learn about Flax Linen and its documentation.

4 Week 2

4.1 Monday, June 22, 2026

Activities

- Start working on the first quantization exercise based on the Mushroom dataset.

Concepts Learned

- One-hot encoding in order to transform the categorical inputs of the mushroom dataset into quantitative attributes using the pandas library from Python.
- Generated a random permutation using JAX in order to randomly divide the dataset into: 70% training samples and 30% testing samples. We do this in order to give the model enough data to learn complex patterns while reserving enough unseen data to reliably evaluate how it will perform.
- Implemented a simple multilayer perceptron (MLP):

```

class MushroomClassifier(nn.Module):
    features: Sequence[int]

    @nn.compact
    def __call__(self, inputs):
        x = inputs
        for i, feat in enumerate(self.features):
            x = nn.Dense(feat, name=f'layer_{i}')(x)
            if i != len(self.features) - 1:
                x = nn.relu(x)

        return x

arch = [16, 2]
model = MushroomClassifier(features=arch)

key = jax.random.PRNGKey(0)
dummy_input = jnp.ones((1, 117))
variables = model.init(key, dummy_input)

```

This is a basic type of feed forward neural network. The MLP consists of an input layer that takes the raw numbers from our data. Then some hidden layers, which are ones responsible for learning the patterns and finally an output layer that produces the final prediction. In our MPL above we find: `arch = [16, 2]` which corresponds to: one hidden layer with 16 neurons and an output layer with 2 neurons (logits) that translate to edible (0) or poisonous (1) mushrooms. Every neuron in one layer links to every neuron in the next and data only moves forward. Each connection has a weight that adjusts how important an input is. In order to add non-linearity, neurons use math formulas like `nn.relu(x)`. Thus, the process is the following: the model makes a guess, checks its error and uses a feedback process to tune its weights.

4.2 Tuesday, June 23, 2026

Activities

- Continue working on the first quantization exercise based on the Mushroom dataset.

Concepts Learned

- Implemented a loss function (formula that measures how far off a ML model's predictions are) using the softmax cross entropy from Optax as well as an accuracy function to measures what percentage of total predictions the model gets correct.
- Set a learning rate, which controls the step size during weight updates, and an optimizer. In our case the Adam optimizer from Optax. The optimizer creates the update rule that calculates how our NN's weights should change during training.
- Built a `def trainstep():` function in order to train our model. The training loop compiles our `trainstep` function and runs for a number of epochs while recording loss, train accuracy and test accuracy.
- Obtained our first results from the original FP32 model:

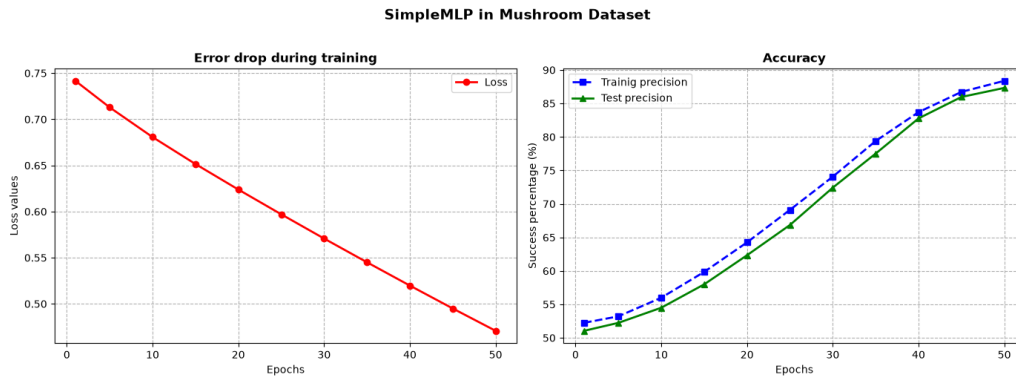


Figure 4: Simple MPL model for `arch = [16, 2]` and `epochs = 50`.

Figure 4 shows graphs for the training accuracy, test accuracy and cross entropy loss. We observe how the error drops and the accuracy improves as we continue to train our model through a number of epochs.

We also calculated confusion matrices so we could further evaluate the classification performance of our model:

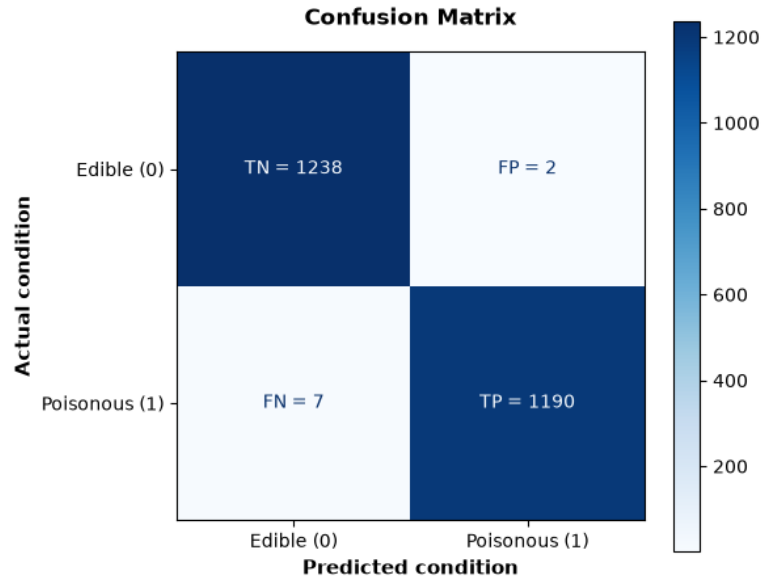


Figure 5: Confusion matrix for `arch = [64, 2]` and `epochs = 100`.

As we can observe from Figure 5, once our model is sufficiently trained, it achieves a small number of misclassified samples.

4.3 Wednesday, June 24, 2026

Activities

- Continue working on the first quantization exercise based on the Mushroom dataset.

Concepts Learned

- After training the FP32 model, its parameters were quantized directly to lower precision formats: INT8, INT4 and INT2 using the Google library Qwix. We first implemented Post-Training Quantization (PTQ) where weight-only quantization (quantifying only the weights to a lower precision format) and DRQ (quantifying the weights offline and the activations dynamically) were the ones tested.

- Did accuracy and latency evaluation for weight-only quantization:

Table 2 summarizes the impact of quantization on the model’s performance. For each data type, the classification accuracy, accuracy loss, inference latency and speed up factor relative to the FP32 model are reported.

Table 2: PTQ performance comparison for `arch = [4, 2]` and epochs = 50.

Data Type	Accuracy (%)	Accuracy Loss (%)	Latency (ms)	Speed up
FP32	87.32	0.00	0.046774	1.00x
INT8	87.32	0.00	0.045655	1.02x
INT4	87.24	0.08	0.043030	1.09x
INT2	51.09	36.23	0.045569	1.03x

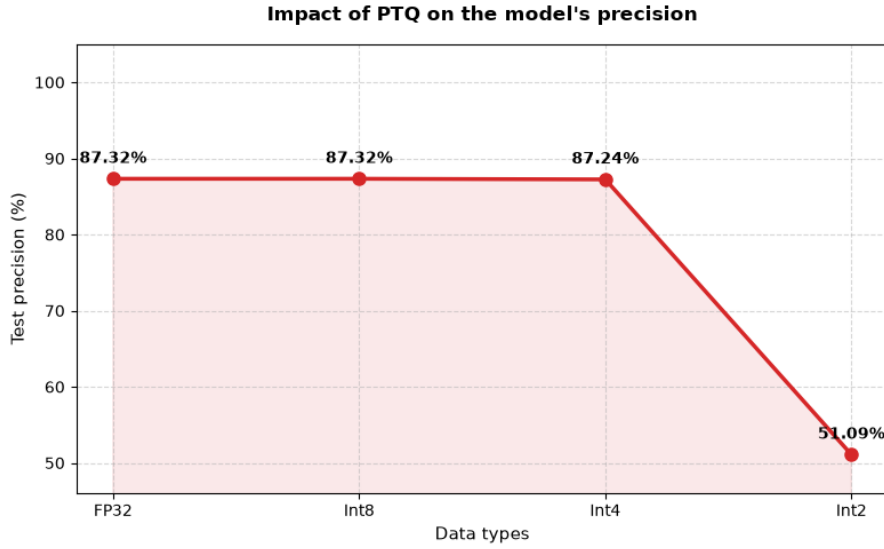


Figure 6: Impact of PTQ on the model’s accuracy for `arch = [4, 2]` and epochs = 50.

From the results obtained above, although at 50 epochs our model is not yet properly trained, data type INT8 and INT4 manage to maintain the original’s model accuracy. It is only when we change to INT2 where we see that significant loss in accuracy. For the latency measurements, we notice how quantization has speeded up the training but not in a significant amount.

- Did accuracy and latency evaluation for DRQ:

Table 3: PTQ performance comparison for `arch = [64, 2]` and epochs = 100.

Data Type	Accuracy (%)	Accuracy Loss (%)	Latency (ms)	Speed up
FP32	99.63	0.00	0.060159	1.00x
INT8	99.67	-0.04	0.093401	0.64x
INT4	99.59	0.04	0.465858	0.13x
INT2	99.02	0.62	0.485931	0.12x

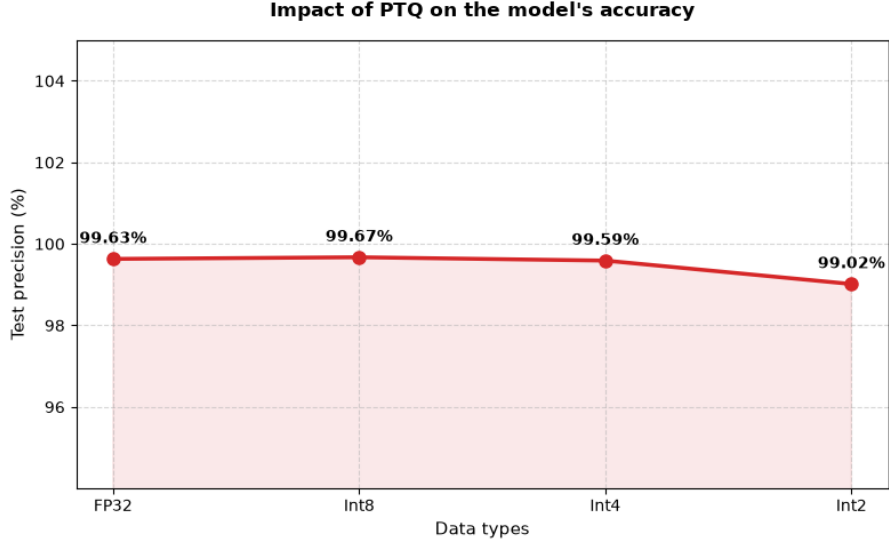
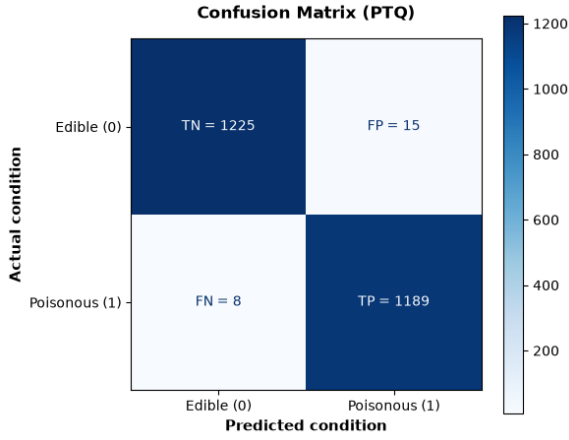


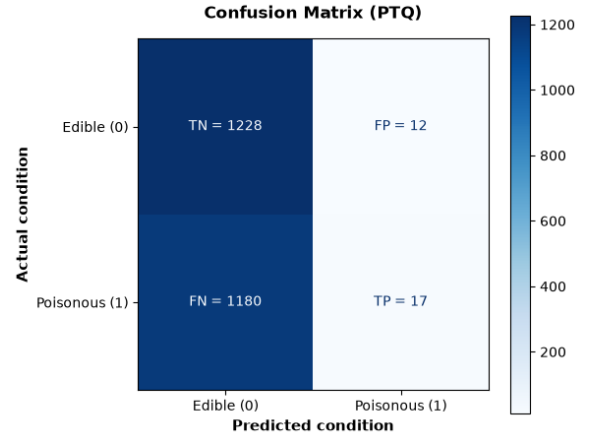
Figure 7: Impact of PTQ on the model's accuracy for `arch` = [64, 2] and epochs = 100.

From the results obtained above, we observe how insignificant the loss in accuracy becomes once our model is properly trained that even INT2 precision becomes a viable option. On the other hand we observe how applying DRQ to our model has increased the inference latency. The model is slower and that might be due to the overhead introduced by dynamic activation quantization.

- Calculated confusion matrices for both weight-only quantization and DRQ:

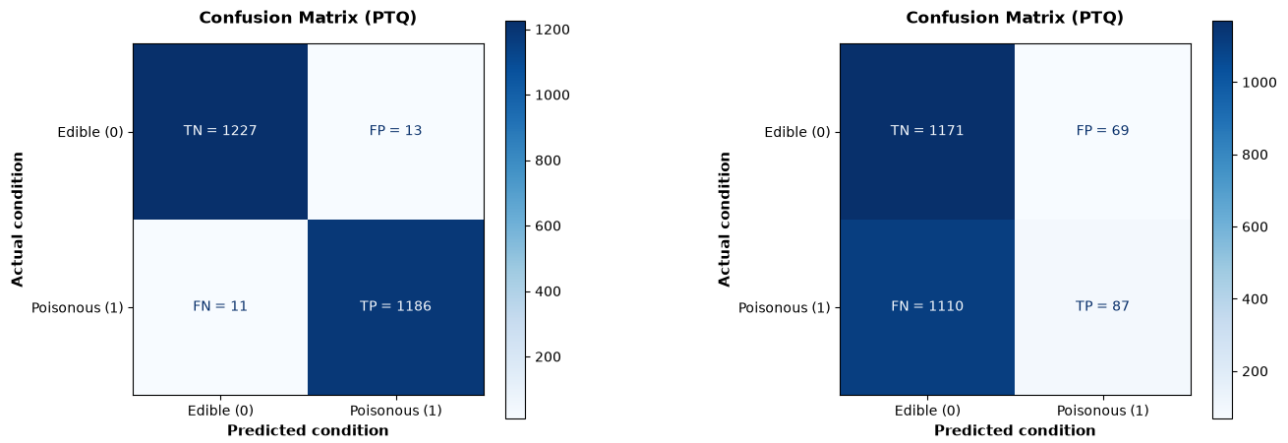


(a) Confusion matrix for weight only quantization with `arch` = [64, 2], epochs = 100 and INT2.



(b) Confusion matrix for weight only quantization with `arch` = [4, 2], epochs = 50 and INT2.

Figure 8: Confusion matrices for the weight only quantification model using data type: INT2.



(a) Confusion matrix for DRQ with $\text{arch} = [64, 2]$, epochs = 100 and INT2.

(b) Confusion matrix for DRQ with $\text{arch} = [4, 2]$, epochs = 50 and INT2.

Figure 9: Confusion matrices for the DRQ model using data type: INT2.

It is clear that once our model is properly trained, the confusion matrices for both weight-only quantization and DRQ produce very similar classification performances, with a small number of misclassified samples even for INT2.

4.4 Thursday, June 25, 2026

Activities

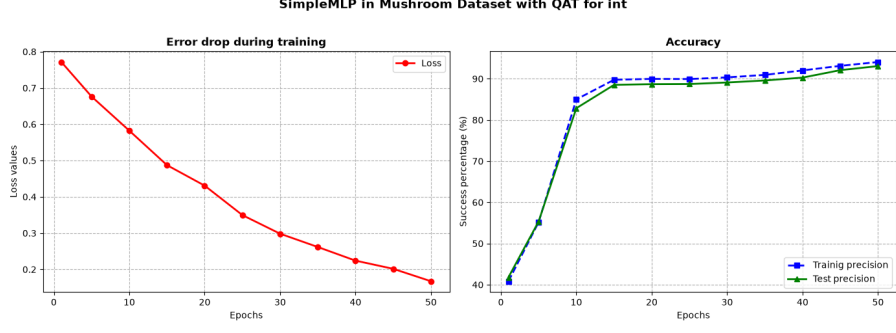
- Finishing working on the first quantization exercise based on the Mushroom dataset.

Concepts Learned

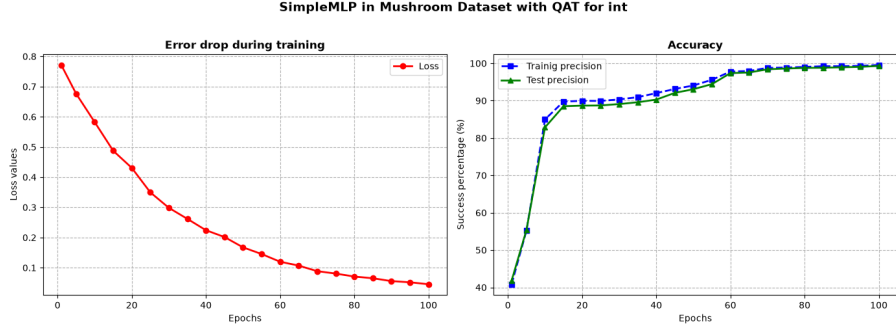
- Learnt to implement QAT. Quantization-Aware Training incorporates quantization effects during training, allowing the optimizer to adapt the parameters to the lower precision formats. The QAT measurements were performed using INT2 and only weight-only quantization was implemented. We used the same training configuration as for the FP32 model.
- Obtained results for the QAT model:

Figures 10 and 11 show the evolution of the loss function and accuracy throughout training for different arch and epoch values.

When comparing the error drop and accuracy evolution figures, we see visual differences between the FP32 model and the QAT model. For the larger architecture: $\text{arch} = [64, 2]$, both models show stable convergence. However, the QAT curves exhibit minor fluctuations in cross-entropy loss due to the gradient noise introduced by simulating lower precision formats during training. For the smaller architecture: $\text{arch} = [4, 2]$ trained with QAT, the error drop and accuracy plots show a fluctuating behaviour full of sharp spikes unlike the monotonic improvement seen in the FP32 model. This pattern is also caused by simulating lower precision formats during training.

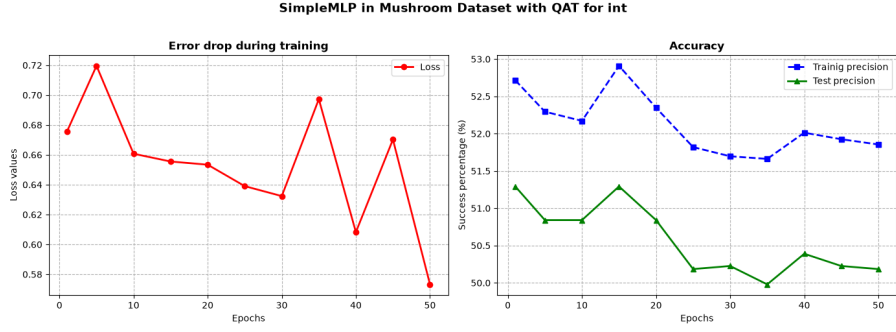


(a) Simple MPL model for $\text{arch} = [64, 2]$ and epochs = 50.

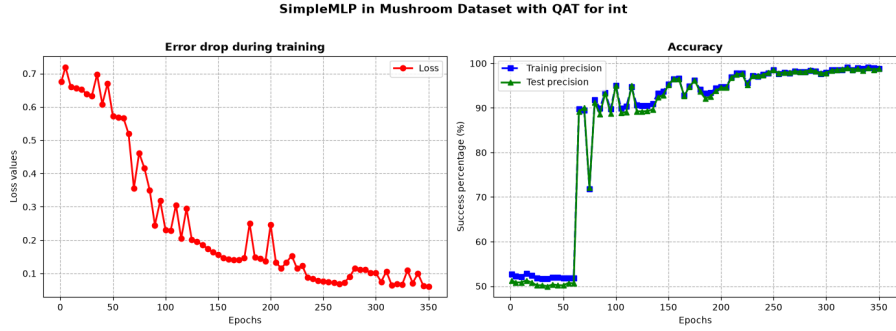


(b) Simple MPL model for $\text{arch} = [64, 2]$ and epochs = 100.

Figure 10: Error and accuracy evolution for the QAT model.



(a) Simple MPL model for $\text{arch} = [4, 2]$ and epochs = 50.



(b) Simple MPL model for $\text{arch} = [4, 2]$ and epochs = 350.

Figure 11: Error and accuracy evolution for the QAT model.

The tables below summarize the impact of quantization on the model’s performance. For different **arch** and epoch values: accuracy and inference latency relative to the FP32 and PTQ (INT2) models are reported.

Table 4: QAT performance comparison for **arch** = [64, 2] and epochs = 50.

Model	Accuracy (%)	Latency (ms)
FP32	96.59	0.062722
QAT INT2	93.07	0.058159
PTQ INT2	96.06	0.061027

Table 5: QAT performance comparison for **arch** = [64, 2] and epochs = 100.

Model	Accuracy (%)	Latency (ms)
FP32	99.63	0.066366
QAT INT2	99.30	0.056841
PTQ INT2	99.06	0.063868

Table 6: QAT performance comparison for **arch** = [4, 2] and epochs = 50.

Model	Accuracy (%)	Latency (ms)
FP32	87.32	0.046774
QAT INT2	50.18	0.052566
PTQ INT2	51.09	0.045569

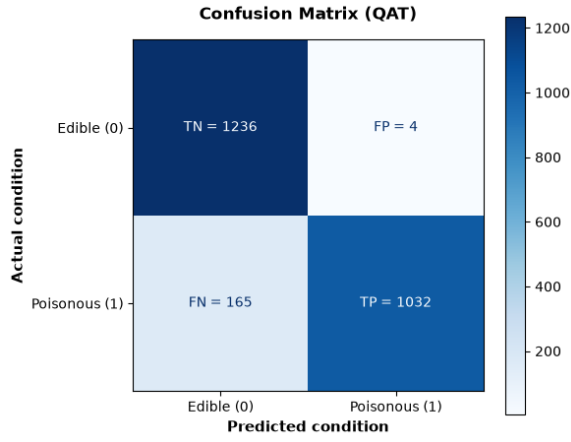
Table 7: QAT performance comparison for **arch** = [4, 2] and epochs = 350.

Model	Accuracy (%)	Latency (ms)
FP32	99.22	0.046260
QAT INT2	98.81	0.046773
PTQ INT2	99.26	0.048495

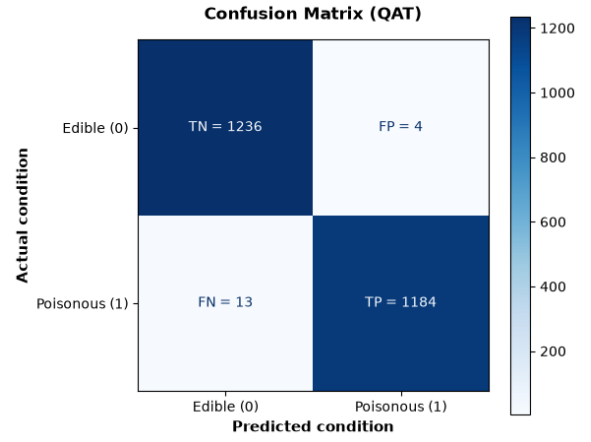
The QAT experiments further show that incorporating quantization during training did not significantly change the model’s final performance when the network had already learned a good FP32 solution. A better improvement from both PTQ and QAT on the latency readings would have been expected, since we are reducing our data types from FP32 to INT2. QAT was expected to perform better than PTQ but here we observe how their results go hand in hand. Nevertheless accuracy is maintained, even for INT2, and latency is slightly improved.

Finally, the confusion matrices obtained for different **arch** and epoch values are shown below in Figures 12 and 13. We again observe how once the models are sufficiently trained, the implementations achieve very similar classification performances. For most of the evaluated models, the number of false negatives goes from 7 to 13. An exception is observed for the smallest QAT model: **arch** = [4,2] trained for 350 epochs, where the number of false negatives increases to 27.

Overall, the experiments show that reducing numerical precision can be a good strategy for maintaining high accuracy while decreasing the computational cost, once the original model has been properly trained.

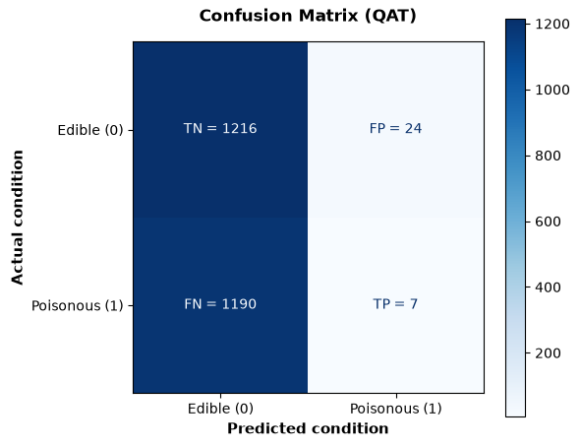


(a) Confusion matrix for $\text{arch} = [64, 2]$ and $\text{epochs} = 50$.

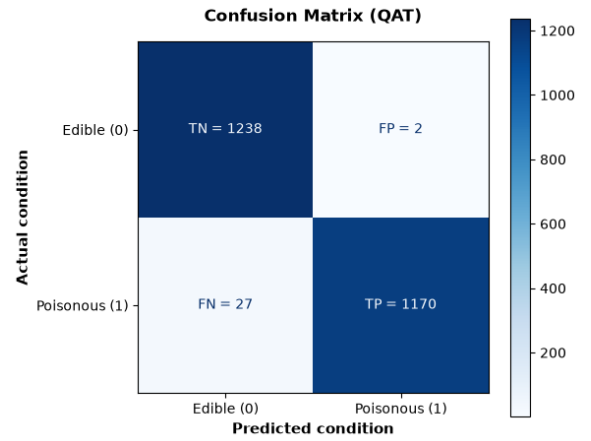


(b) Confusion matrix for $\text{arch} = [64, 2]$ and $\text{epochs} = 100$.

Figure 12: Confusion matrices for QAT model.



(a) Confusion matrix for $\text{arch} = [4, 2]$ and $\text{epochs} = 50$.



(b) Confusion matrix for $\text{arch} = [4, 2]$ and $\text{epochs} = 350$.

Figure 13: Confusion matrices for QAT model.

4.5 Friday, June 26, 2026

Activities

- Beginning of next quantization project: investigate the effects of weight only quantization on a neural-network quantum state (NQS) that represented the ground state of the one dimensional transverse-field Ising model (TFIM).

Concepts Learned

- Familiarized myself with the NetKet library in order to implement the Monte Carlo algorithm for the exercise.

5 Week 3

5.1 Monday, June 29, 2026

Activities

- Continuing with the new quantization exercise for the TFIM.

Concepts Learned

- Did research about the TFIM as well as some NetKet tutorials.

5.2 Tuesday, June 30, 2026

Activities

- Continuing with the new quantization exercise for the TFIM.

Concepts Learned

- Started the implementation:

We used the one dimensional transverse-field Ising model with periodic boundary conditions.

The Hamiltonian is given by:

$$H = \Gamma \sum_{i=1}^N \sigma_i^x + V \sum_{i=1}^N \sigma_i^z \sigma_{i+1}^z$$

where σ^x and σ^z denote the Pauli matrices, $\Gamma = -1$ is the transverse magnetic field and $V = -1$ is the nearest-neighbour interaction strength. We considered a system with $N = 20$ spins-1/2 corresponding to a ferromagnetic Ising chain.

The Hamiltonian is constructed using the operator framework provided by NetKet:

```
Gamma = -1
H = sum([Gamma * sigmax(hi, i) for i in range(N)])

V = -1
H += sum([V * sigmaz(hi, i) @ sigmaz(hi, (i + 1) % N) for i in range(N)
])
```

Although the neural-network approach is variational, an exact reference solution is available for the system size considered here. Thus, the sparse Hamiltonian matrix was diagonalized using the SciPy library from Python.

We note that: the lowest eigenvalue corresponds to the exact ground-state energy, while the associated eigenvector is later used to evaluate the quality of the neural-network approximation.

The quantum wavefunction is represented by a fully-connected neural network implemented using Flax Linen. Given a spin configuration:

$$\mathbf{s} = (s_1, s_2, \dots, s_N),$$

the neural network outputs the logarithm of the corresponding wavefunction amplitude.

The chosen architecture is a simple multilayer perceptron (MLP) consisting of one hidden layer with 150 neurons followed by a ReLU activation function:

```
class IsingMLP(nn.Module):

    dtype: any = jnp.float32

    @nn.compact
    def __call__(self, x):

        x = nn.Dense(
            features=150,
            use_bias=True,
            dtype=self.dtype
        )(x)

        x = nn.relu(x)

        x = nn.Dense(
            features=1,
            use_bias=False,
            dtype=self.dtype
        )(x)

        return jnp.squeeze(x, axis=-1)
```

The first dense layer extracts nonlinear features from the spin configurations, while the second layer produces a scalar value associated with each many-body basis state.

The neural network is incorporated into NetKet through the `MCState` class, which combines the variational model with a Monte Carlo sampler:

```
sampler = nk.sampler.MetropolisLocal(hi)

vstate = nk.vqs.MCState(
    sampler,
    model,
    n_samples=1024
)
```

Sampling is performed using the Metropolis local algorithm, where each Monte Carlo move attempts to flip a single spin. The resulting configurations are used to estimate expectation values of quantum observables through stochastic sampling instead of explicitly summing over the entire Hilbert space.

After optimization, several quantities are evaluated in order to characterize the quality of the variational wavefunction.

The primary observable is the expectation value of the Hamiltonian:

$$E = \langle \psi | H | \psi \rangle,$$

which is directly minimized during training.

To compare the variational solution with the exact diagonalization result, the relative energy error is computed as:

$$\epsilon_E = \frac{|E - E_{\text{ED}}|}{|E_{\text{ED}}|}.$$

In addition, the overlap between the neural-network wavefunction and the exact ground state is calculated according to:

$$O = |\langle \psi_{\text{ED}} | \psi_{\text{NN}} \rangle|^2.$$

An overlap close to one indicates that both wavefunctions describe nearly the same quantum state.

Finally, nearest-neighbour spin correlations are evaluated through:

$$C_{xx} = \sum_i \sigma_i^x \sigma_{i+1}^x,$$

providing an additional physical observable beyond the ground-state energy. Comparing these correlation functions across different numerical precisions allows us to assess whether quantization preserves not only the variational energy but also the physical properties of the quantum state.

- Obtained first results for the TFIM:

Exact energy (ED): -25.490989686364756 J

Optimized energy and relative error: $-25.428 \pm 0.015 J$

Overlap between FP32 y ED: 0.99627

$X - X$ Correlator (ED): 10.852249

Figure 14 shows how much it takes our FP32 model to converge to the desired energy and it compares it with the result obtained from exact diagonalization.

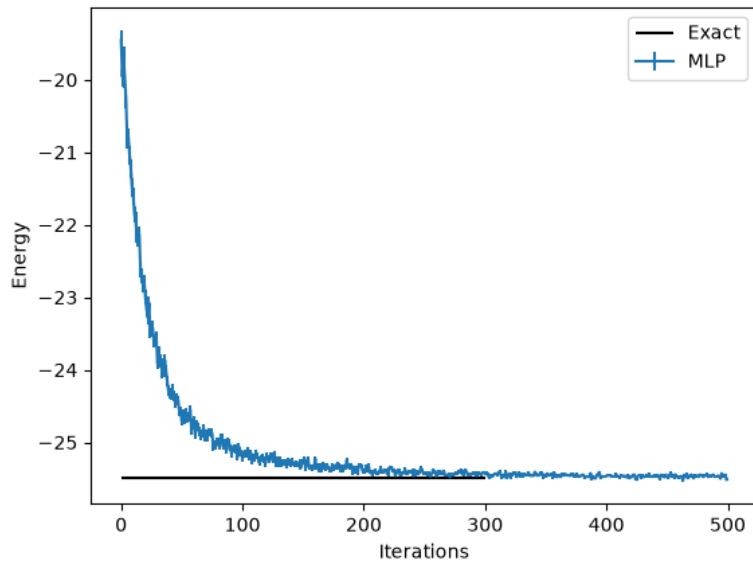


Figure 14: Energy convergence comparison between MLP and ED.

5.3 Wednesday, July 01, 2026

Activities

- Continuing with the new quantization exercise for the TFIM.

Concepts Learned

- Once the FP32 model for the TFIM was properly implemented using JAX, Flax Linen and the NetKet library we can begin our quantization quest.
- Used the Qwix library to implement PTQ on our model for INT8, INT4 and INT2.
- Learnt that JAX supports FP16 numerical precision and therefore implemented quantization for such data type as well (without Qwix).
- Obtained results:

Table 8: PTQ performance comparison for the 1D TFIM using different data types.

Data Type	Energy (H)	Relative Error	Correlator	Corr Error	Overlap	Latency (ms)	Speed Up
FP32	-25.427646	0.002485	10.818816	0.033432	0.996269	0.602828	1.00x
FP16	-25.472749	0.0007155	10.765304	0.086945	1.000504	0.336029	1.79x
INT8	-25.464798	0.001027	10.726541	0.125708	0.997199	0.234925	2.57x
INT4	-25.372614	0.004644	11.746746	0.894497	0.958133	0.305106	1.98x
INT2	-22.182162	0.129804	16.589086	5.736837	0.200938	0.665848	0.91x

As we can observe from Table 8, changing to FP16 within JAX is a very viable option that does not compromise the quality of our results and indeed decreases the computational cost as seen from the latency readings.

Regarding PTQ using Qwix, we see how using INT8 quantization for our model is a very viable option. Not only is the fastest implementation but its results for the overlap and energy are better (closer to ED) than the ones obtained for the FP32 model. Notice how decreasing to INT4 and especially to INT2 starts giving far off results.

Figure 15 shows how the different errors measured, change for the different data types.

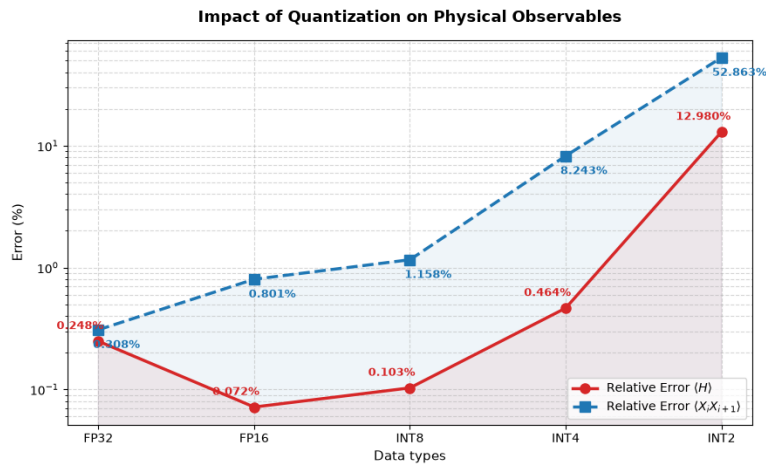


Figure 15: Impact of quantization on error measurements.

It is logical that we observe how the error increases as we reduce numerical precision.

5.4 Thursday, July 02, 2026

Activities

- Continuing with the new quantization exercise for the TFIM.

Concepts Learned

- Started implementing QAT on our exercise.
- Changed to $N = 10$ and obtained some new metrics:

Figure 16 shows the evolution of the energy convergence for the different data types against the ED result.

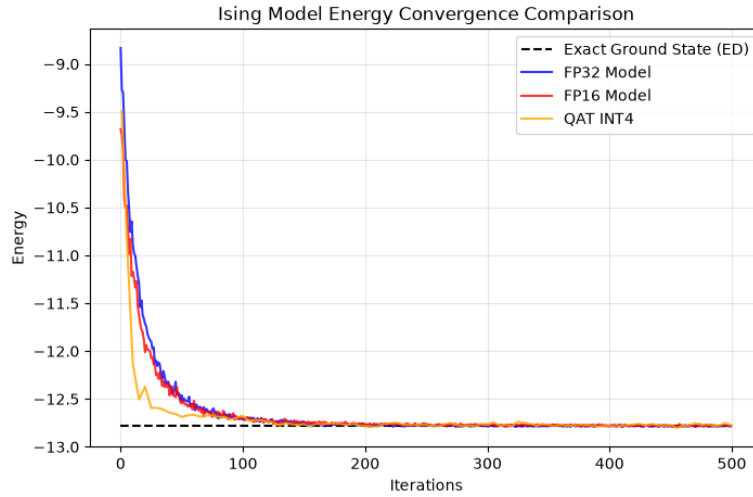


Figure 16: Energy convergence comparison between models.

A training time evolution was also monitored for the QAT model as seen in Figure 17:

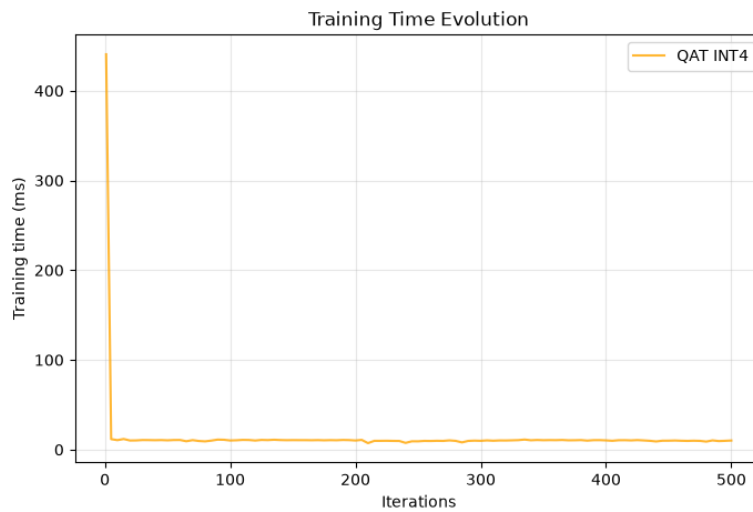


Figure 17: Training time evolution for the QAT INT4 model.

We can observe that the model converges quite quickly, being: 10.131836 ms the training time for the last iteration.

We also obtained a table with the following data:

Table 9: QAT performance comparison for the 1D TFIM using different data types.

Data Type	Energy (H)	Relative Error	Correlator	Corr Error	Overlap	Latency (ms)	Speed Up
FP32	-12.781754	0.000246	5.425310	0.012575	0.990462	0.574493	1.00x
FP16	-12.766040	0.001476	5.389978	0.019006	0.965745	0.193748	2.97x
INT4	-12.758444	0.002070	5.190808	0.055255	0.995119	0.359624	1.60x

Where the results for the exact diagonalization were:

Exact energy (ED): $-12.784906 J$.

Exact correlator (ED): 5.494405.

Comparing the results obtained from ED against the ones in Table 9, we observe again how quantization using QAT becomes a viable choice if we want to decrease the computational cost of our simulation but still maintain accurate results.

5.5 Friday, July 03, 2026

Activities

- Continuing with the new quantization exercise for the TFIM.

Concepts Learned

- Implemented new piece of code in order to observe the overlap between the ED wavefunction and the neural-network wavefunction for different values of V in the ferromagnetic phase.
- Obtained the following graph:

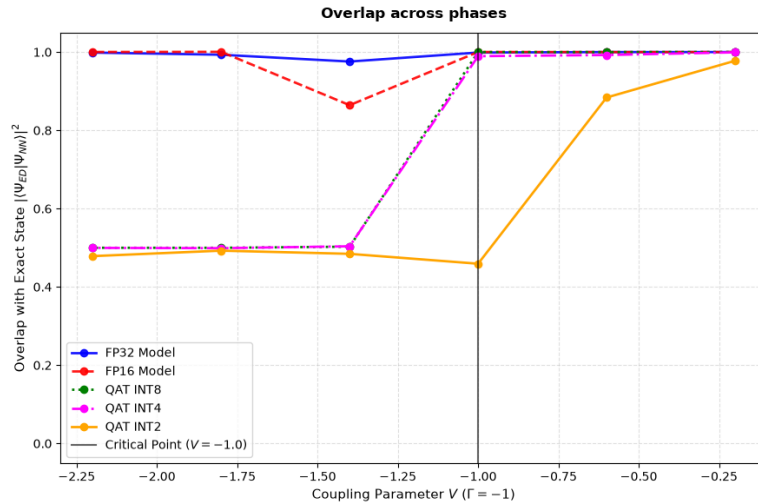


Figure 18: Overlap for different data types across the ferromagnetic phase.

As we can see from Figure 18, the overlap measurements are very off for the QAT data types: INT8, INT 4 and INT2. Nevertheless, we observe how they all fall off at around 0.5. This points out that we ought to take into account the $\mathbb{Z}_2$ symmetry (the energy of the system does not change when flipping every single spin along the z-axis at the same time) of the TFIM in our implementation.

6 Week 4

6.1 Monday, July 06, 2026

Activities

- Continuing with the new quantization exercise for the TFIM.

Concepts Learned

- Changed how we defined our MLP in order to take into account the $\mathbb{Z}_2$ symmetry:

```
class IsingMLP(nn.Module):
    dtype: any = jnp.float32

    @nn.compact
    def __call__(self, x):
        dense1 = nn.Dense(features=150, use_bias=True, dtype=self.dtype)
        dense2 = nn.Dense(features=1, use_bias=False, dtype=self.dtype)

        out_x = dense1(x)
        out_x = nn.relu(out_x)
        out_x = dense2(out_x)

        out_minus_x = dense1(-x)
        out_minus_x = nn.relu(out_minus_x)
        out_minus_x = dense2(out_minus_x)

        return jnp.squeeze(0.5 * (out_x + out_minus_x), axis=-1)
```

Instead of relying on data augmentation or forcing the model to learn the symmetry through training gradients, this architecture builds the symmetry directly into the model graph by construction. We evaluate Branch 1 (x) by passing the original input x through `dense1` and applying a relu activation. We then project through `dense2` to produce an unsymmetrized scalar output, `outx`. Now we look at Branch 2 ($-x$) and evaluate the exact same network with shared parameters using the spin-flipped state $-x$. We then pass $-x$ through `dense1`, apply relu and project through `dense2` to get `outminusx`. Finally, we compute the arithmetic mean of both branches: $f_{\text{sym}}(x) = \frac{1}{2} [f_{\text{raw}}(x) + f_{\text{raw}}(-x)]$.

- Implemented translational symmetry as well but did not use it.
- Played with different learning schedules.
- Built the new QAT model for the TFIM project taking into account the $\mathbb{Z}_2$ symmetry.

6.2 Tuesday, July 07, 2026

Activities

- Finishing the new quantization exercise for the TFIM.

Concepts Learned

- Obtained results for the TFIM where $N = 15$ using QAT and taking into account the $\mathbb{Z}_2$ symmetry:

Results for Exact diagonalization had been:

Exact energy (ED): -19.133544 J .

Exact correlator (ED): 8.165457.

Table 10: QAT performance comparison for the 1D TFIM with $\mathbb{Z}_2$ symmetry using different data types.

Data Type	Energy (H)	Relative Error	Correlator	Corr Error	Overlap	Latency (ms)	Speed Up
FP32	-19.132604	4.917699e-05	8.204049	0.004726	0.999006	0.051557	1.00x
FP16	-19.102007	0.0016483	8.139875	0.003133	0.999778	0.043761	1.18x
INT4	-19.123969	0.000500	8.065403	0.012253	0.998307	0.054505	0.95x

As we can observe from Table 10, the QAT model is able to reproduce a good value for the energy, correlator and overlap. Nevertheless we find how it becomes slower than our original FP32 model and that might be due to the computational costs that quantization introduces during training.

We also produced figures for the energy convergence for each data type and for the training time evolution:

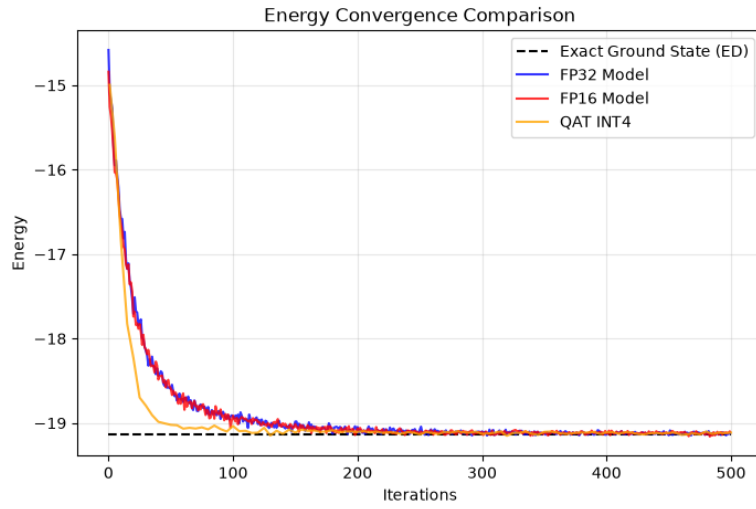


Figure 19: Energy convergence comparison between models.

From Figure 19 we can observe how the FP32 and FP16 models converge simultaneously while the QAT INT4 model might converge even faster.

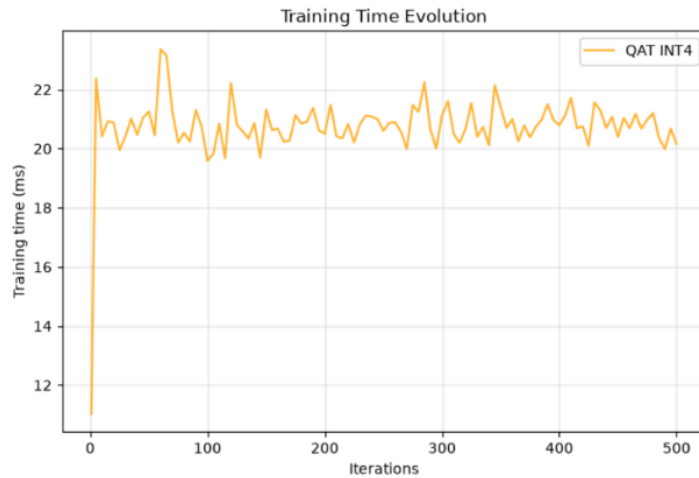
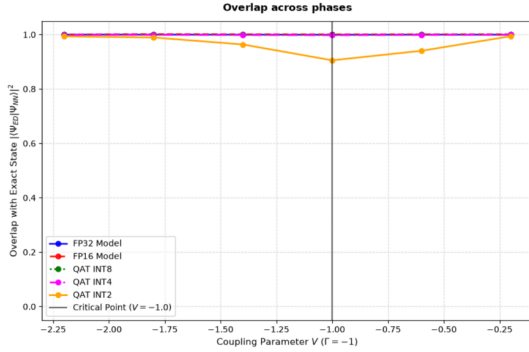


Figure 20: Training time evolution for the QAT INT4 model with $\mathbb{Z}_2$ symmetry.

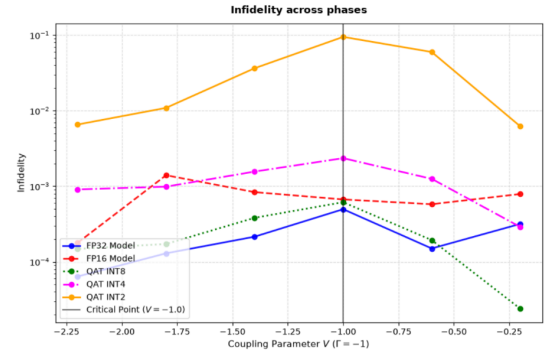
From Figure 20 we can observe that the model converges quite quickly, being: 20.162821 ms the

training time for the last iteration. Since in this new MLP we are doing two calculations at the same time, one for x and another one for $-x$, the training time doubles.

Finally we observe below how taking into account the $\mathbb{Z}_2$ symmetry has solved our problems for the overlap measurements:



(a) Overlap for different data types across the ferromagnetic phase.



(b) Infidelity error for the different overlap measurements.

Figure 21: Overlap for different data types across the ferromagnetic phase when taking into account $\mathbb{Z}_2$ symmetry.

From Figure 21a we observe how the overlap measurement for all data types but INT2 lay at the 1.0 line. That is a great result that validates our quantization project. Even the INT2 precision achieved good accuracy for the overlap measurement. It aligns with theory that the weakest point lies at exactly $V = -1$ since that is where we find the quantum phase transition.

Furthermore, with Figure 21b we are able to zoom-in and observe the error between the overlap measurements. It is logic for the INT2 measurement to have the biggest error.

6.3 Wednesday, July 08, 2026

Activities

- Did an analysis of the projects done so far with my supervisors.
- Chose to start implementing the concepts learned for the ViT from the reference data.

Concepts Learned

- Started to study the working code again in order to understand how to implement the ViT onto my PTQ/QAT simulation for the TFIM.

6.4 Thursday, July 09, 2026

Activities

- Begin working on implementing the ViT into my TFIM simulation.

Concepts Learned

- Researched how ViT worked.
- Continued studying the working code from the reference data.

6.5 Friday, July 10, 2026

Activities

- Continue working on implementing the ViT into my TFIM simulation.

Concepts Learned

- Continued researching how ViT worked.
- Continued studying the working code from the reference data.

7 Week 5

7.1 Monday, July 13, 2026

Activities

- Continue working on implementing the ViT into my TFIM simulation.

Concepts Learned

- Started implementing the ViT into the TFIM simulation.
- Used the same Hamiltonian and implemented a wrapper in order to enforce $\mathbb{Z}_2$ spin-flip symmetry.
- Used same Metropolis sampler from working code as well as same rule for inverting 75% of the spins in the model.
- Used same logic from the working code in order to build a learning schedule for that helps implement a better learning rate.
- Implemented the ViT for the FP32 data type and got first results:

Energy from FP32 model: -12.789378 J .

Exact energy (ED): -12.784906 J .

V-score from FP32 model: 0.0003141.

- Struggled to implement the FP16 data type.

7.2 Tuesday, July 14, 2026

Activities

- Continue working on implementing the ViT into my TFIM simulation.

Concepts Learned

- Achieved the FP16 data type implementation.
- Obtained data for the FP16 model:

Energy from FP16 model: -12.793378 J .

Exact energy (ED): -12.784906 J .

V-score from FP16 model: 0.001647.

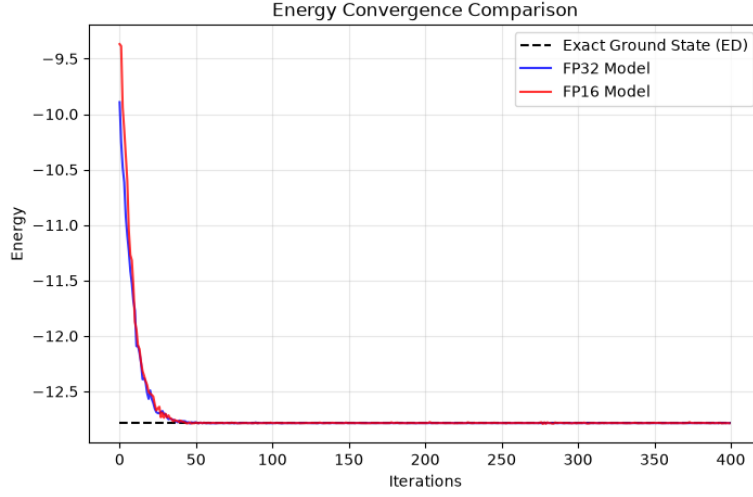


Figure 22: Energy convergence comparison between models using the ViT.

As we can observe from Figure 22 both models are able to converge quickly and simultaneously.

7.3 Wednesday, July 15, 2026

Activities

- Continue working on implementing the ViT into my TFIM simulation.

Concepts Learned

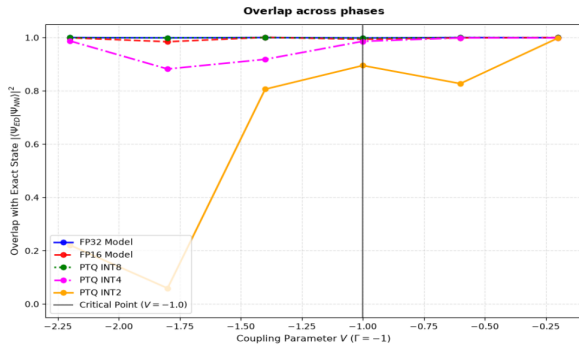
- Implemented PTQ into the ViT simulation for the TFIM.
- Obtained results for PTQ:

Table 11: PTQ performance comparison for the 1D TFIM (ViT) with $\mathbb{Z}_2$ symmetry using different data types.

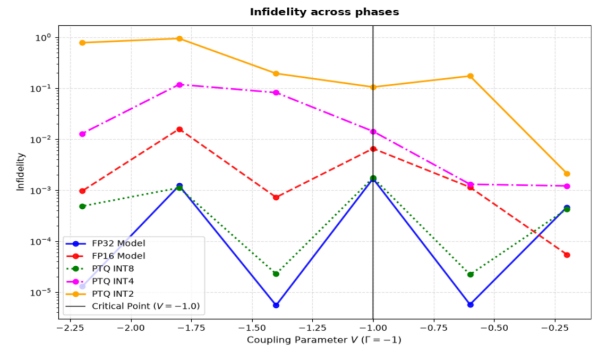
Data Type	Energy (H)	Relative Error	Overlap	V-score	Latency (ms)	Speed Up
FP32	-12.788336	0.000268	0.998721	0.000333	0.657260	1.00x
FP16	-12.790295	0.000421	0.996735	0.000465	0.347614	1.89x
INT8	-12.784411	3.875470e-05	0.998784	0.000047	0.541015	1.21x
INT4	-12.508474	0.0216218	0.944502	0.007123	0.648870	1.01x
INT2	-12.283744	0.039200	0.901989	0.020836	0.731366	0.90x

As we can observe from Table 11 changing to FP16 and INT8 numerical precision is a viable option for our model since the speed up factor starts to be significant while maintaining accurate results. For INT4 and INT2, we observe how the data starts to be off from the ED result obtained above and there is no significant improvement in the inference speed. If so the INT2 model is slower.

We have also obtained figures for the overlap during different values of V :



(a) Overlap for different data types across the ferromagnetic phase.



(b) Infidelity error for the different overlap measurements.

Figure 23: Overlap for the PTQ implementation of the TFIM using the ViT across the ferromagnetic phase when taking into account $\mathbb{Z}_2$ symmetry.

From Figure 23a we observe how the overlap measurements produce good results for the FP32 model, the FP16 model and the PTQ INT8 model. The INT4 data type manages to maintain itself in the $[0.8 - 1.0]$ interval for the overlap values and holds its shape at the critical point. Nevertheless, we observe how INT2 is completely lost and not valid for our implementation.

From Figure 23b we observe how the errors obtained align with the overlap measurements obtained in Figure 23a.

It is worth noting that the PTQ INT8 model performed better than the FP16 model for the overlap measurements.

7.4 Thursday, July 16, 2026

Activities

- Continue working on implementing the ViT into my TFIM simulation.

Concepts Learned

- Implemented QAT into the ViT simulation for the TFIM.
- Obtained first results for the QAT implementation:

Energy from QAT INT4: $-12.808723 J$.

Exact energy (ED): $-12.784906 J$.

We note that the simulations for the ViT had been all run for $N = 10$.

We have also obtained figures for the converge of the energy:

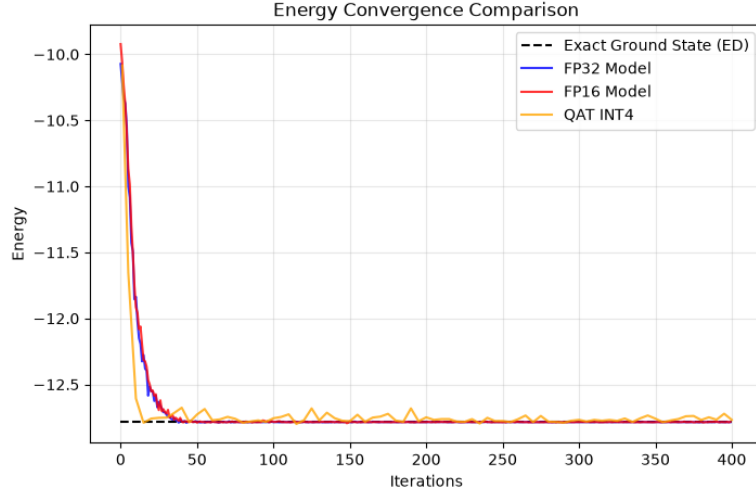


Figure 24: Energy convergence comparison between models using the ViT for QAT.

We observe how again the FP32 and FP16 models converge simultaneously while the QAT INT4 model is found with more oscillations during its convergence period due to the incorporation of quantization during training.

The training time evolution was also monitored:

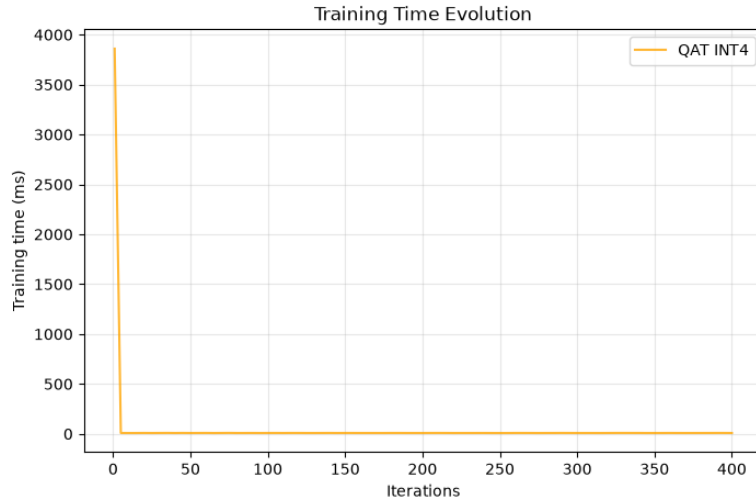


Figure 25: Training time evolution for the ViT QAT INT4 model with $\mathbb{Z}_2$ symmetry.

From figure 25 we observe how the model's training converges quickly. We found: 8.440733 ms to be the training time in the last iteration.

7.5 Friday, July 17, 2026

Activities

- Finish working on implementing the ViT into my TFIM simulation.

Concepts Learned

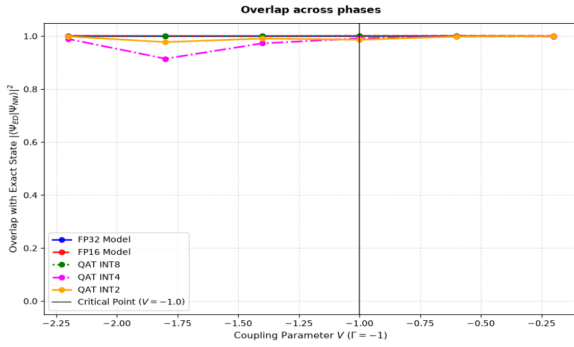
- Obtained last results for the ViT QAT implementation:

Table 12: QAT performance comparison for the 1D TFIM (ViT) with $\mathbb{Z}_2$ symmetry using different data types.

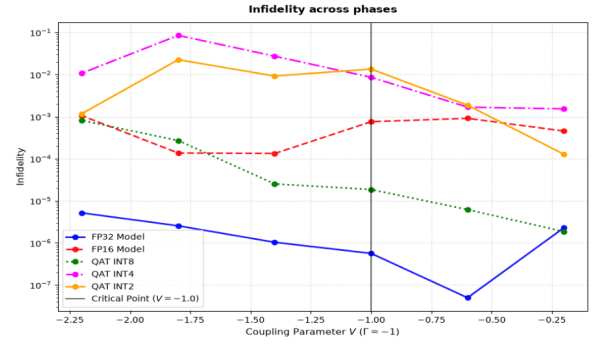
Data Type	Energy (H)	Relative Error	Overlap	V-score	Latency (ms)	Speed Up
FP32	-12.784949	3.329011e-06	0.999998	0.000101	2.736734	1.00x
FP16	-12.785411	3.949304e-05	1.000039	0.000040	0.316519	8.65x
INT4	-12.783785	8.770330e-05	0.992943	0.000071	0.778351	3.52x

From Table 12 we observe how quantization is a great viable option for our implementation. That is because it is able to reproduce accurate results while decreasing the computational cost. We observe speed up values of 8.65x and 3.52x.

Furthermore, we were also able to obtain figures for the overlap:



(a) Overlap for different data types across the ferromagnetic phase.



(b) Infidelity error for the different overlap measurements.

Figure 26: Overlap for the QAT implementation of the TFIM using the ViT across the ferromagnetic phase when taking into account $\mathbb{Z}_2$ symmetry.

As we can observe from Figure 26a, the overlap measurements for the different data types are good. We find the QAT INT4 and QAT INT2 models being a little off but within the $[0.9 - 1.0]$ interval. We observe how INT2 performs better than INT4 and from Figure 26b we notice how INT8 has a smaller error than FP16.

Overall we see how it is possible to implement a quantization exercise for the TFIM using the ViT. We note how the results obtained using the MLP for the TFIM were more accurate since the architecture of the MLP is much more simpler than the one used to implement the ViT.

8 Week 6

8.1 Monday, July 20, 2026

Activities

- Discussed all the projects done during the internship with my supervisors.
- Talked about how simple it would be to implement the long-range TFIM into my existing simulations since we would only need to change the Hamiltonian in the code.
- Shared that a continuation of the project would be to integrate extreme quantization (as in e.g. BinaryAttention).